第31章 詳解

31-E01 Euclid 的證書與輸入長度

推導

391=1・299+92；299=3・92+23；92=4・23+0。故gcd=23。回推 23=299−3・92=299−3(391−299)=4・299−3・391。因gcd不是1，299沒有mod391 inverse；若有299u−391v=1，左側被23整除，矛盾。

對a≥b>0，r=a mod b。若r≤b/2已減半；若r>b/2，下一餘數b mod r=b−r<b/2。所以每兩個遞除至少把次參數減半，β位輸入O(β)次。一次β位長除法仍可能需$O(\beta^2)$bit，上界相乘可得$O(\beta^3)$，不能把大整數除法當1bit操作。

結論與檢核

Bezout係數以(391,299)次序為(−3,4)，驗算−1173+1196=23；沒有模逆不是算法失敗，而是必要互質條件不成立。

31-E02 全部模線性方程解

推導

gcd(24,60)=12，12|36。除12得2x≡3(mod5)，$2^{-1}\equiv 3$，所以x≡9≡4(mod5)。模60全部12解是4,9,14,19,24,29,34,39,44,49,54,59。每次加5，原乘積加120，是60倍數；兩解之差由5|2(x−x_0)與互質消去得5|(x−x_0)，所以沒有漏。

右側35不被12整除，無解。0x≡0(mod9)對x=0,⋯,8皆成立，壓縮成(x_0=0,step=1,count=9)。一般0x≡0(mod n)有n個代表，若n為β位，列全n項本身就可能對β指數大；壓縮形式只需O(β)位元。

結論與檢核

有解條件、數目12與間距5都必要。不可在mod60直接對24乘不存在的inverse。

31-E03 CRT 的建構與相容性

推導

M=4・9・5=180，M_i=(45,20,36)，各inverse mod(4,9,5)為(1,5,1)，selector為(45,100,36)。重建x=1・45+2・100+3・36=353≡173(mod180)。檢查173 mod4=1、mod9=2、mod5=3。

第二系統gcd(8,12)=4，差7−3=4相容。x=3+8k，代mod12得8k≡4，除4得2k≡1(mod3)，k≡2，所以x=19 mod24，周期lcm=24。若餘數改6，差3不被4整除；任何共同解差的整除條件被違反，因此無解。

第31章 詳解 （續）

結論與檢核

第一系統唯一模180；第二相容系統唯一模24，在模96下有四個代表19,43,67,91。不能把原模數乘積當最小周期。

31-E04 Order、Primitive Root 與不同的群

推導

2^k mod11，k=0,⋯,10為1,2,4,8,5,10,9,7,3,6,1，首次回1在10，所以order10、generator；7在k=7，故ind(7)=7 mod10。

U(8)={1,3,5,7}，每個odd x都有$x^2 \equiv 1 \pmod 8$，所以order皆≤2，沒有order4元素，不能生成四個元素。它仍是群，有逆不表示有generator。

$2^{10}=1024 \equiv 56=1+5\cdot 11 \pmod{121}$，不等於1。mod121的order必被10整除（投影mod11），也整除φ121=110，所以只能10或110；前者已排除，故order110。其實(1+55)^11≡1(mod121)，與Euler界一致。2本身就能作lift，不需改成2+11。

結論與檢核

有限交換群不一定cyclic；odd prime power的lift仍需檢查非平凡的p^2層，而不是由一次小模數測試直接推大模數order。

31-E05 左到右與右到左模冪

推導

左到右初始化r=1：bit1→平方1、乘7得7；bit0→$7^2 \equiv 3$；bit0→$3^2=9$；bit1→$9^2 \equiv 12$、乘7≡15；bit1→$15^2 \equiv 18$、乘7≡11。前綴1,2,4,9,19，答案11。

右到左令r=1 mod n、a=A mod n、b=B。每輪若b odd則r←ra mod n，接著a←a^2 mod n、b←⌊b/2⌋。若b=2q，更新後$r(a^2)^q = r\,a^b$；若b=2q+1，更新後$(ra)(a^2)^q = r\,a^b$。故r·a^b≡A^B保持，b逐半終止於0，r即答案。

b=0不進迴圈，回1 mod n；n=1回0。乘方不需inverse，所以例如A=6,n=15仍正確。若要B<0，應另檢gcd(A,n)=1求逆，不能讓負值整除循環不終止。

結論與檢核

$7^{19} \bmod 23 = 11$。兩種掃描的不變量不同，不能拿左掃的「已讀前綴」解釋右掃尚未處理的b。

第31章 詳解 （續）

31-E06 RSA 的非 Unit 訊息與可操作性

推導

n=91、φ=6・12=72，1=29・5−2・72，所以d=29。m=7，加密$7^2\equiv 49$、$7^4\equiv 35$，$7^5\equiv 35\cdot 7\equiv 63 \pmod{91}$。解密$63^2\equiv 56$、$63^4\equiv 42$、$63^8\equiv 35$、$63^{16}\equiv 42$；29=16+8+4+1，依序42・35≡14、14・42≡42、42・63≡7。

雖gcd(7,91)=7，mod7兩邊訊息皆0；mod13則7是unit，ed=145≡1(mod12)，由Fermat得$7^{145}\equiv 7$。CRT保證mod91唯一還原7。

2為unit，c'=c・2^5 mod91是E(2m mod91)，因$m^5\cdot 2^5=(2m)^5$。對此m，c'=63・32≡14，解密回14。這表明裸RSA可被乘法操作，並非證安全；知道p,q能求d，也不能因此說逆RSA已被證與分解等價。

結論與檢核

公鑰(91,5)、d=29，非unit訊息照樣可解密。演算法正確性、整數分解難度與完整密碼方案安全性是三個不同命題。

31-E07 Strong Liar 根數的完整核對

推導

65=5・13，64=2^6・1，所以s=6,d=1。5−1=2^2・1、13−1=2^2・3，t=(2,2)、u=(1,3)、D=(1,1)、τ=2、r=2。公式L=(1+1+4)・1=6。

a^d=1只給a=1。j=0的a=−1給64。j=1需$a^2=-1 \bmod 65$：mod5根2,3，mod13根5,8。四個CRT組合得到8,18,47,57；平方均≡64。它們與1,64不同，已列全6個。

抽樣[2,63]有62個base，排除1,64後剩4個liars，通過比例4/62=2/31<1/4。base2平方鏈為2,4,16,61,16,61，沒有64，判合數；最後再平方也不成1，亦違反Fermat。

結論與檢核

全域strong liars為{1,8,18,47,57,64}。根數各分量獨立，但MR接受要求同一j層的符號同步，不可把各自任何層的−1全部組合。

31-E08 隨機測試與事後機率

推導

4^14=268435456<10^9、4^15=1073741824>10^9，所以至少15輪。
ε=4^(−15)=1/1073741824。先驗π=1/100，Bayes上界
ε・99/(1+99ε)=99/(1073741824+99)，約9.22・10^{-8}。它大於固定合數接受率的界，因合數先驗是質數的99倍。

第31章 詳解 （續）

若每次同一base2，事件完全相同，不可把機率相乘。2047=23・89，2^{11}=2048≡1 mod2047，而d=1023=11・93，所以2^d=1，每次都通過；重複任意輪仍接受此合數。

結論與檢核

15輪的保證需要獨立均勻抽樣。固定n的錯誤率與觀察pass後的合數率分母不同，後者必附候選分布；以上π是題給假設。

31-E09 Pollard Rho 的分量碰撞

推導

n=91時第一輪x=f(2)=5、y=f(f(2))=f(5)=26，diff21，gcd(21,91)=7；5≡26 mod7，但mod13的5與0不同，所以得到真因數7，另一因數13。

n=15,x_0=0，$f=x^2+2$：首輪x=f(0)=2、y=f(2)=6，diff4，gcd=1；第二輪x=f(2)=6，y=f(f(6))=f(8)=6，diff0，gcd(0,15)=15。這只回傳整個n，沒有得到proper divisor，必須restart或回報本次未找到。

一般函數圖有tail長μ、cycle長λ。第k輪位置是x_k,x_(2k)；k≥μ後兩者都在cycle，相差k modλ。至多再λ輪遇k≡0(modλ)，故O(μ+λ)輪、常數個值儲存。模某因數的碰撞正確性確定，但要把μ+λ估成O(√p)，另假設此多項式模p行為近似random mapping；固定多項式不受該最壞保證。

結論與檢核

合法成功必1<d<n。d=n只指出兩個值在全部模n分量同時碰撞，無法拆n。預算到期也只能回未找到，不可因此判質數。

第31章 詳解 （續）

原創驗算程式附錄: number_theory_checks.py

```
"""Check the number-theory proofs on exact small instances."""

from math import gcd


def egcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = egcd(b, a % b)
    q = a // b
    return g, y1, x1 - q * y1


def modexp(base, exponent, modulus):
    assert exponent >= 0 and modulus > 0
    value = 1 % modulus
    prefix = 0
    for bit in bin(exponent)[2:]:
        value = value * value % modulus
        prefix *= 2
        if bit == "1":
            value = value * base % modulus
            prefix += 1
        assert value == pow(base, prefix, modulus)
    return value


def strong_liars(n):
    d, s = n - 1, 0
    while d % 2 == 0:
        d //= 2
        s += 1
    liars = []
    for base in range(2, n - 1):
        value = pow(base, d, n)
        if value in (1, n - 1):
            liars.append(base)
            continue
        for _ in range(s - 1):
            value = value * value % n
            if value == n - 1:
                liars.append(base)
                break
    return liars
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第31章 詳解 （續）

```
def rsa_round_trip():
    p, q = 5, 11
    n, phi = p * q, (p - 1) * (q - 1)
    e, d = 3, 27
    assert e * d % phi == 1
    for message in range(n):
        cipher = pow(message, e, n)
        assert pow(cipher, d, n) == message


def run():
    g, x, y = egcd(30, 18)
    assert g == gcd(30, 18) == 6
    assert 30 * x + 18 * y == 6
    assert [(value, (3 * value - 6) % 12) for value in (2, 6, 10)] == [
        (2, 0),
        (6, 0),
        (10, 0),
    ]
    assert modexp(5, 13, 11) == 4
    assert modexp(5, 29, 11) == 9
    assert modexp(5, 0, 11) == 1
    rsa_round_trip()

    composite_cases = 0
    for n in range(9, 501, 2):
        if all(n % divisor for divisor in range(3, int(n**0.5) + 1, 2)):
            continue
        liars = strong_liars(n)
        assert len(liars) * 4 <= n - 1, (n, liars)
        composite_cases += 1
    print(
        "PASS gcd=6 modexp=4,9,1 rsa_residues=55 "
        f"strong_liar_bound_cases={composite_cases}"
    )


if __name__ == "__main__":
    run()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第31章 詳解 （續）

原創驗算程式附錄: number_theory_lab.py

```
"""Exact educational algorithms and traces. No cryptographic deployment API."""
from math import gcd,isqrt
def extended(a,b):
    sa=-1 if a<0 else 1;sb=-1 if b<0 else 1;a=abs(a);b=abs(b)
    r0,r1=a,b;x0,x1=1,0;y0,y1=0,1;trace=[]
    while r1:
        q=r0//r1;trace.append((r0,r1,q,r0-q*r1))
        r0,r1=r1,r0-q*r1;x0,x1=x1,x0-q*x1;y0,y1=y1,y0-q*y1
    return r0,x0*sa,y0*sb,trace
def linear(a,b,n):
    if n<1:raise ValueError('modulus must be positive')
    g,u,_,_=extended(a,n)
    if b%g:return []
    step=n//g;x=(u*(b//g))%step
    return [x+k*step for k in range(g)]
def crt(pairs):
    x=0;m=1
    for a,n in pairs:
        if n<1:raise ValueError('positive moduli required')
        g,u,_,_=extended(m,n)
        if (a-x)%g:return None
        k=((a-x)//g*u)%(n//g);x+=m*k;m*=n//g;x%=m
    return x,m
def modular_power(a,b,n):
    if n<1 or b<0:raise ValueError('n>=1 and exponent>=0 required')
    a%=n;r=1%n;v=0;trace=[]
    for bit in (bin(b)[2:] if b else ''):
        before=r;square=r*r%n;r=square
        if bit=='1':r=r*a%n
        v=2*v+int(bit);trace.append((int(bit),v,before,square,r))
    return r,trace
def strong_pass(n,a):
    if n<5 or n%2==0:raise ValueError('odd n>=5 required')
    if not 1<=a<n:raise ValueError('base in1..n-1 required')
    d=n-1;s=0
    while d%2==0:s+=1;d//=2
    x=modular_power(a,d,n)[0];chain=[x]
    if x in (1,n-1):return True,chain
    for _ in range(s-1):
        x=x*x%n;chain.append(x)
        if x==n-1:return True,chain
    return False,chain
def probable_prime(n,bases):
    if n<2:return False
    if n in (2,3):return True
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第31章 詳解 （續）

```
    if n%2==0:return False
    for a in bases:
        if not 2<=a<=n-2:raise ValueError('random test base must be in2..n-2')
        if not strong_pass(n,a)[0]:return False
    return True
def trial_factors(n):
    if n<2:raise ValueError('n>=2 required')
    ans=[];p=2
    while p*p<=n:
        e=0
        while n%p==0:n//=p;e+=1
        if e:ans.append((p,e))
        p+=1
    if n>1:ans.append((n,1))
    return ans
def liar_count_formula(n):
    if n<5 or n%2==0:raise ValueError('odd n>=5')
    d=n-1;s=0
    while d%2==0:s+=1;d//=2
    factors=trial_factors(n);ts=[];D=1
    for p,_ in factors:
        u=p-1;t=0
        while u%2==0:t+=1;u//=2
        ts.append(t);D*=gcd(d,u)
    tau=min(s,*ts);r=len(factors)
    return D*(1+sum(2**(j*r) for j in range(tau)))
def rho(n,x0=2,c=1,budget=10000):
    if n<=1:raise ValueError('n>1')
    if n==2:return None,[]
    if n%2==0:return 2,[]
    f=lambda x:(x*x+c)%n;x=y=x0%n;trace=[]
    for i in range(1,budget+1):
        x=f(x);y=f(f(y));d=gcd(abs(x-y),n);trace.append((i,x,y,abs(x-y),d))
        if 1<d<n:return d,trace
        if d==n:return None,trace
    return None,trace
if __name__=='__main__':
    assert rho(2)==(None,[])
    for candidate in (3,4,6,9,15,25,49,8051):
        factor,_=rho(candidate)
        assert factor is None or (1<factor<candidate and candidate%factor==0)

print('Bezout',extended(252,198));print('linear',linear(18,30,42));print('CRT',crt([(2,3),
(4,5),(1,7)]))

print('modexp',modular_power(5,13,11));print('MR341',strong_pass(341,2));print('RSA',pow(11,7
,187),pow(88,23,187));print('rho',rho(8051))
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第31章 詳解 （續）

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study